

SafeD Translate Bot - Project Development Roadmap

Project Overview

Total Estimated Hours: 280 hours **Team Composition:**

- 2 Backend Developers
- 1 Frontend Developer
- 1 QA Tester

Project Duration: ~8 weeks (based on 35 hours/week with 4-person team)

Team Resource Allocation

Backend Team (2 developers - 160 hours total)

- Backend Developer 1: 80 hours
- Backend Developer 2: 80 hours

Frontend Team (1 developer - 80 hours total)

- Frontend Developer: 80 hours

QA Team (1 tester - 40 hours total)

- QA Tester: 40 hours
-

Phase 1: Planning & Architecture (Week 1) - 35 hours

Backend Team (20 hours)

- **Requirements Analysis (8 hours)**
 - Review current translation bot architecture
 - Document API requirements and integration points
 - Database schema design
 - Third-party service integration planning (OCR, translation APIs)
- **Technical Architecture Design (12 hours)**
 - Design microservices architecture
 - API endpoint specifications
 - Data flow diagrams
 - Security and authentication strategy
 - Scalability planning

Frontend Team (10 hours)

- **UI/UX Planning (10 hours)**
 - User flow diagrams
 - Wireframes for bot interactions
 - Admin dashboard mockups
 - User experience optimization

QA Team (5 hours)

- **Test Strategy Planning (5 hours)**
 - Test plan creation
 - Define test cases structure
 - Setup test environment requirements
 - Automation framework selection

Phase 2: Core Backend Development (Weeks 2-4) - 85 hours

Backend Team (70 hours)

Backend Developer 1 (35 hours)

- **Translation Service Enhancement** (15 hours)
 - Implement multi-language support improvements
 - Optimize translation API integration (existing pattern with axios)
 - Add translation caching mechanism
 - Error handling and retry logic
- **OCR Service Improvement** (12 hours)
 - Enhance Tesseract.js integration
 - Support for multiple image formats
 - PDF text extraction optimization
 - Document preprocessing pipeline
- **File Processing Service** (8 hours)
 - DOCX file handling enhancement (mammoth.js)
 - XLSX file processing (existing xlsx library)
 - PDF parsing improvements (pdfjs-dist)
 - File validation and sanitization

Backend Developer 2 (35 hours)

- **API Development** (15 hours)
 - RESTful API endpoints (Express.js)
 - Webhook management (existing webhooks module)
 - Request/response optimization
 - API documentation (OpenAPI/Swagger)
- **Bot Core Enhancement** (12 hours)
 - Telegram Bot API integration improvements
 - Command handler optimization
 - Inline query enhancements
 - Callback query processing
- **Database & Storage** (8 hours)
 - Setup cloud storage integration (Ali OSS existing)
 - User data management
 - Translation history storage

- Session management

Frontend Team (10 hours)

- **Bot Interface Design** (10 hours)
 - Telegram bot UI improvements
 - Keyboard layouts optimization (existing keyboards.js)
 - Inline button designs
 - User feedback mechanisms

QA Team (5 hours)

- **Test Case Development** (5 hours)
 - Unit test cases for backend services
 - Integration test scenarios
 - Test data preparation
-

Phase 3: Advanced Features & Frontend (Weeks 5-6) - 70 hours

Backend Team (35 hours)

Backend Developer 1 (18 hours)

- **Advanced Translation Features** (10 hours)
 - Language auto-detection
 - Batch translation processing
 - Translation quality improvement
 - Custom dictionary support
- **Analytics & Monitoring** (8 hours)
 - Usage statistics tracking
 - Performance monitoring
 - Error logging and reporting
 - User behavior analytics

Backend Developer 2 (17 hours)

- **Admin Features** (10 hours)
 - Admin command handlers
 - Configuration management
 - User management system
 - System health monitoring
- **Integration Enhancements** (7 hours)
 - Third-party API integrations
 - Webhook optimization
 - Real-time updates
 - Push notification system

Frontend Team (25 hours)

- **Admin Dashboard** (15 hours)
 - Dashboard UI development
 - Real-time statistics display
 - User management interface
 - System configuration panel
- **User Interface Enhancements** (10 hours)
 - Improved bot conversation flow
 - Rich message formatting
 - Progress indicators
 - Error message improvements

QA Team (10 hours)

- **Testing Execution - Phase 1** (10 hours)
 - Unit testing execution
 - Integration testing
 - Bug reporting and tracking
 - Test documentation
-

Phase 4: Integration & Testing (Week 7) - 50 hours

Backend Team (25 hours)

- **System Integration** (15 hours)
 - End-to-end integration testing
 - Performance optimization
 - Security hardening
 - API rate limiting implementation
- **Bug Fixes & Refinement** (10 hours)
 - Address QA findings
 - Code optimization
 - Memory leak fixes
 - Error handling improvements

Frontend Team (10 hours)

- **UI/UX Refinement** (10 hours)
 - User feedback implementation
 - Interface polish
 - Accessibility improvements
 - Mobile optimization

QA Team (15 hours)

- **Comprehensive Testing** (15 hours)
 - Full regression testing
 - Performance testing
 - Security testing
 - User acceptance testing preparation
 - Test report generation

Phase 5: Deployment & Documentation (Week 8) - 40 hours

Backend Team (20 hours)

- **Production Preparation** (12 hours)
 - Production environment setup
 - CI/CD pipeline configuration (GitLab CI/Jenkins)
 - Docker deployment optimization
 - Database migration scripts
 - Backup and recovery procedures
- **API Documentation** (8 hours)
 - Complete API documentation
 - Developer guides
 - Integration examples
 - Troubleshooting guides

Frontend Team (10 hours)

- **Final UI Polish** (5 hours)
 - Final user interface adjustments
 - Cross-platform testing
 - Performance optimization
- **User Documentation** (5 hours)
 - User guides
 - FAQ documentation
 - Video tutorials preparation
 - Help system integration

QA Team (10 hours)

- **Final Validation** (6 hours)
 - Production environment testing
 - Smoke testing
 - Final bug verification
- **Documentation** (4 hours)

- Test summary report
 - Known issues documentation
 - Release notes contribution
-

Risk Management

Technical Risks

1. **Third-party API limitations** - Mitigation: Implement fallback mechanisms
2. **Performance bottlenecks** - Mitigation: Early performance testing
3. **OCR accuracy issues** - Mitigation: Multiple OCR engine support

Resource Risks

1. **Developer availability** - Mitigation: Cross-training team members
2. **Scope creep** - Mitigation: Strict change management process
3. **Integration delays** - Mitigation: Buffer time in schedule

Mitigation Strategies

- Weekly progress reviews
 - Daily standups
 - Continuous integration and testing
 - Regular stakeholder communication
-

Deliverables

Phase 1

-  Technical architecture document
-  API specifications
-  UI/UX wireframes
-  Test plan

Phase 2

- ☒ Core backend services
- ☒ API endpoints
- ☒ Database schema
- ☒ Bot core functionality

Phase 3

- ☒ Advanced features
- ☒ Admin dashboard
- ☒ Analytics system
- ☒ Enhanced UI

Phase 4

- ☒ Integrated system
- ☒ Bug fixes
- ☒ Test reports
- ☒ Performance optimizations

Phase 5

- ☒ Production deployment
- ☒ Complete documentation
- ☒ Training materials
- ☒ Final release

Success Metrics

Performance Metrics

- Translation response time: < 2 seconds
- OCR processing time: < 5 seconds per image
- System uptime: > 99.5%
- Concurrent user support: 1000+ users

Quality Metrics

- Code coverage: > 80%

- Bug density: < 1 bug per 1000 lines of code
- User satisfaction: > 4.5/5.0
- API response success rate: > 99%

Business Metrics

- User adoption rate
- Daily active users
- Translation volume
- Cost per translation

Budget Breakdown (280 hours)

Role	Hours	Percentage
Backend Developer 1	80	28.6%
Backend Developer 2	80	28.6%
Frontend Developer	80	28.6%
QA Tester	40	14.2%
Total	280	100%

Weekly Timeline Summary

Week	Phase	Focus	Hours
1	Planning	Architecture & Design	35
2-4	Development	Core Backend & Initial Frontend	85

5-6	Enhancement	Advanced Features & Frontend	70
7	Integration	Testing & Bug Fixes	50
8	Deployment	Production & Documentation	40

Communication Plan

Daily

- Team standup meetings (15 minutes)
- Slack/Teams updates

Weekly

- Sprint planning (1 hour)
- Progress review with stakeholders (30 minutes)
- Sprint retrospective (45 minutes)

Bi-weekly

- Demo sessions for investors/management
 - Technical review meetings
-

Technology Stack

Backend

- **Runtime:** Node.js 18
- **Framework:** Express.js
- **Bot Framework:** node-telegram-bot-api
- **OCR:** Tesseract.js
- **Document Processing:** mammoth, pdfjs-dist, xlsx
- **HTTP Client:** axios, node-fetch

- **Storage:** Ali OSS

DevOps

- **Containerization:** Docker
- **CI/CD:** GitLab CI, Jenkins
- **Deployment:** Docker Compose

Quality Assurance

- **Testing Framework:** TBD (Jest/Mocha recommended)
 - **API Testing:** Postman/Newman
 - **Load Testing:** Apache JMeter
-

Next Steps

1. Immediate Actions (Week 1)

- Kick-off meeting with full team
- Setup development environments
- Access to required tools and services
- Repository access and branch strategy

2. Week 1 Deliverables

- Completed architecture document
- Finalized API specifications
- Approved UI/UX designs
- Test strategy document

3. Stakeholder Reviews

- Week 2: Architecture review
 - Week 4: Mid-development demo
 - Week 6: Feature complete review
 - Week 8: Final acceptance
-

Last Updated: 2026-04-24 Document Owner: Project Manager Status: Draft - Awaiting Approval